

AIE683 Machine Learning

Week 10: Unsupervised Learning: Dimensionality Reduction & Clustering

Dr. Hakan Emekci

Hacettepe University, Graduate School of Informatics

Plan for today

Block	Content	Time
1	Motivating case; PCA; PCA vs LDA	45 min
2	Manifold learning: t-SNE, UMAP; Try it 1	45 min
	Break	10 min
3	k-means, agglomerative, DBSCAN, HDBSCAN, mixture models	50 min
4	Evaluating clusterings, MLflow sweep; Try it 2	30 min

Notebook: `lab.ipynb` in this week's folder (worked examples, MLflow sweep, both exercises).

A case with no labels

Wholesale customers (UCI / OpenML 1511)

- 440 business clients of a food distributor in Portugal
- annual spend in 6 categories: fresh, milk, grocery, frozen, detergents/paper, delicatessen
- no "segment" column exists

Questions marketing asks:

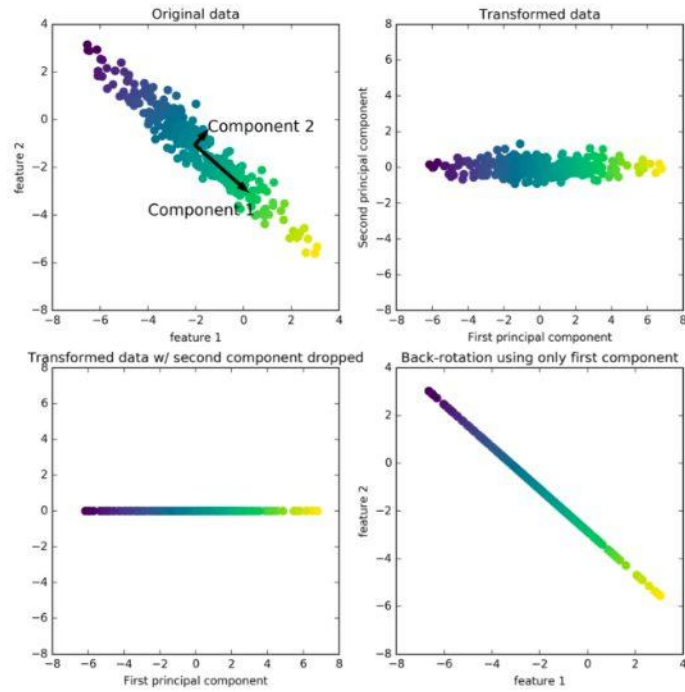
- Are there distinct kinds of customers?
- Can we see the customer base on one screen?
- Which customers are unusual?

Other places the same questions come up:

- single-cell RNA-seq: 20 000 genes per cell, cell types unknown
- image or product catalogs: find near-duplicates, browse by similarity
- sensor fleets: which machines behave alike?
- survey data: respondent profiles

Part 1: Dimensionality Reduction

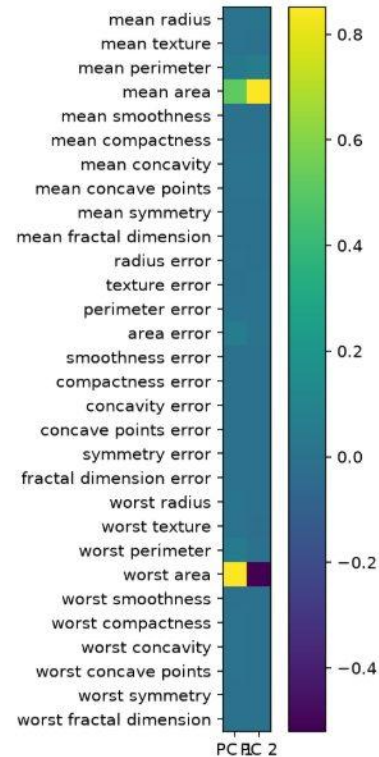
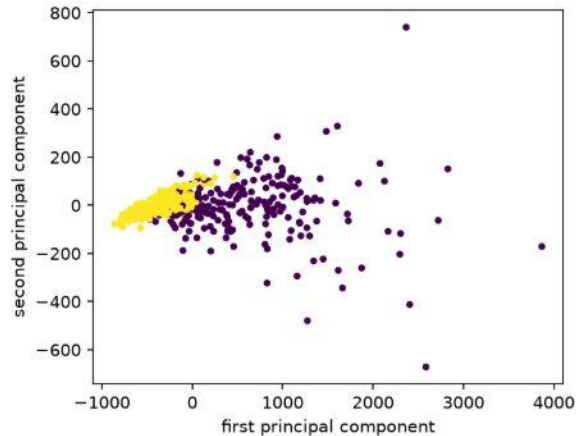
Principal Component Analysis



PCA for Visualization

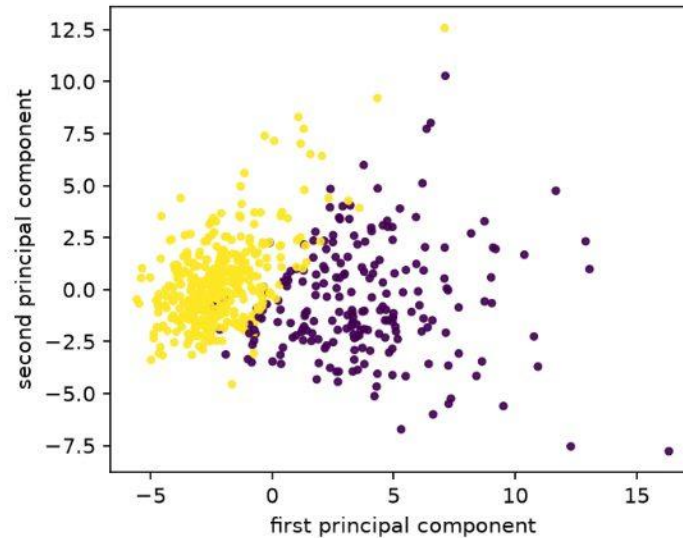
```
from sklearn.decomposition import PCA
print(cancer.data.shape)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(cancer.data)
print(X_pca.shape)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=cancer.target)
plt.xlabel("first principal component")
plt.ylabel("second principal component")
components = pca.components_
plt.imshow(components.T)
plt.xticks(range(len(cancer.feature_names)), cancer.feature_names)
plt.colorbar()
```

(569, 30)
(569, 2)



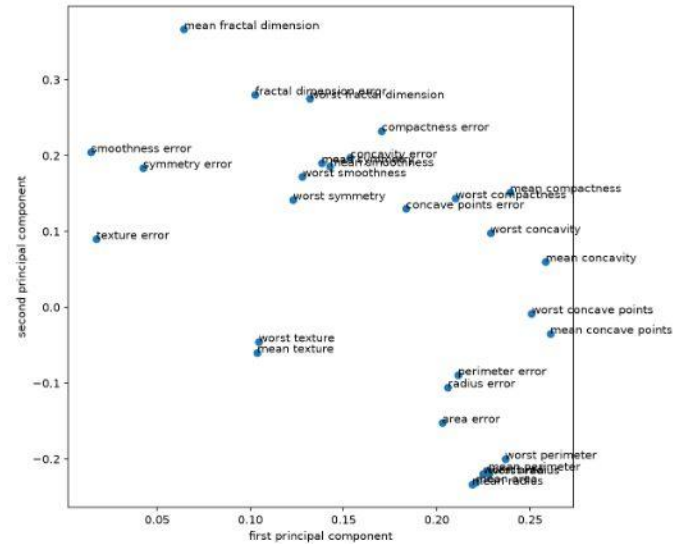
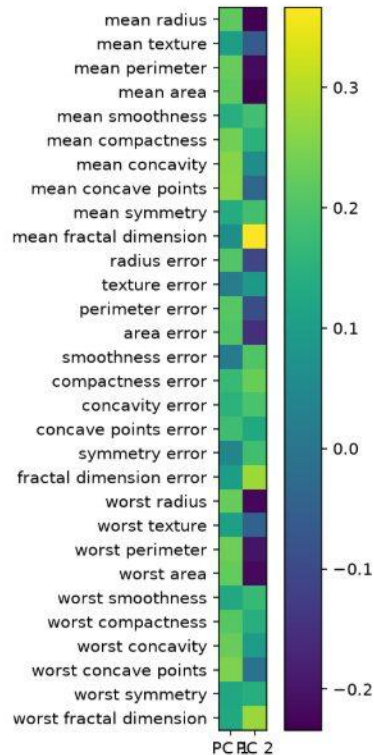
Scaling!

```
pca_scaled = make_pipeline(StandardScaler(), PCA(n_components=2))  
X_pca_scaled = pca_scaled.fit_transform(cancer.data)  
plt.scatter(X_pca_scaled[:, 0], X_pca_scaled[:, 1], c=cancer.target, alpha=.9)  
plt.xlabel("first principal component")  
plt.ylabel("second principal component")
```



Inspecting components

```
components = pca_scaled.named_steps['pca'].components_  
plt.imshow(components.T)  
plt.xticks(range(len(cancer.feature_names)), cancer.feature_names)  
plt.colorbar()
```



PCA for regularization

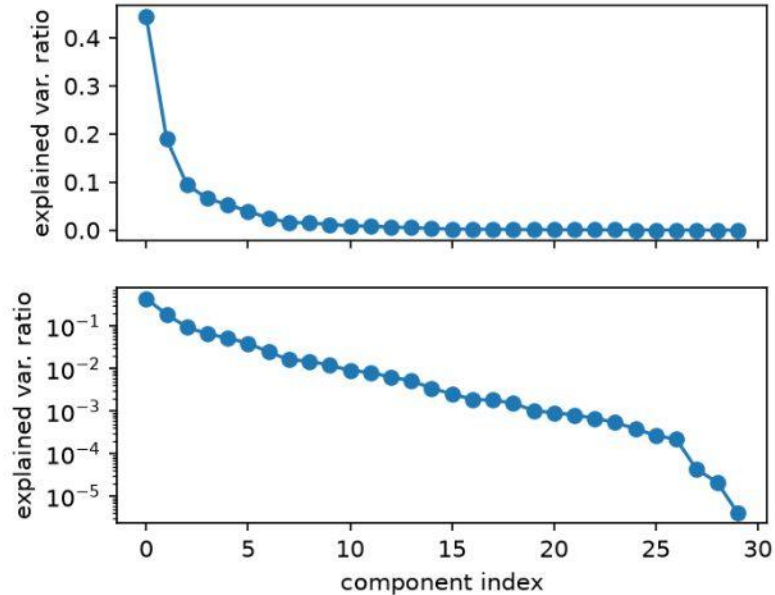
```
X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target,
                                                    stratify=cancer.target, random_state=0)
lr = make_pipeline(StandardScaler(), LogisticRegression(C=10000, max_iter=10000))
lr.fit(X_train, y_train)
print(lr.score(X_train, y_train), lr.score(X_test, y_test))
```

1.000 0.930

```
pca_lr = make_pipeline(StandardScaler(), PCA(n_components=2), LogisticRegression(C=10000))
pca_lr.fit(X_train, y_train)
print(pca_lr.score(X_train, y_train), pca_lr.score(X_test, y_test))
```

0.960 0.923

Variance covered



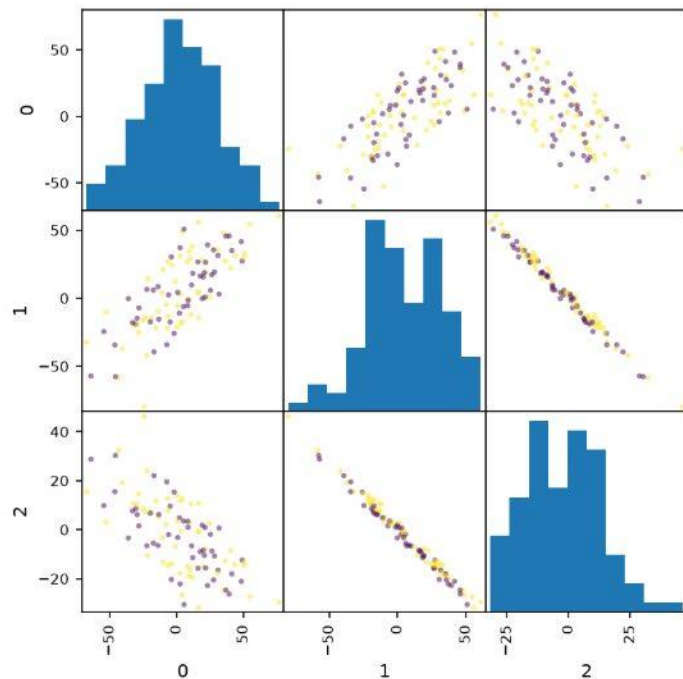
```
pca_lr = make_pipeline(StandardScaler(), PCA(n_components=6), LogisticRegression(C=10000))  
pca_lr.fit(X_train, y_train)  
print(pca_lr.score(X_train, y_train), pca_lr.score(X_test, y_test))
```

0.981 0.958

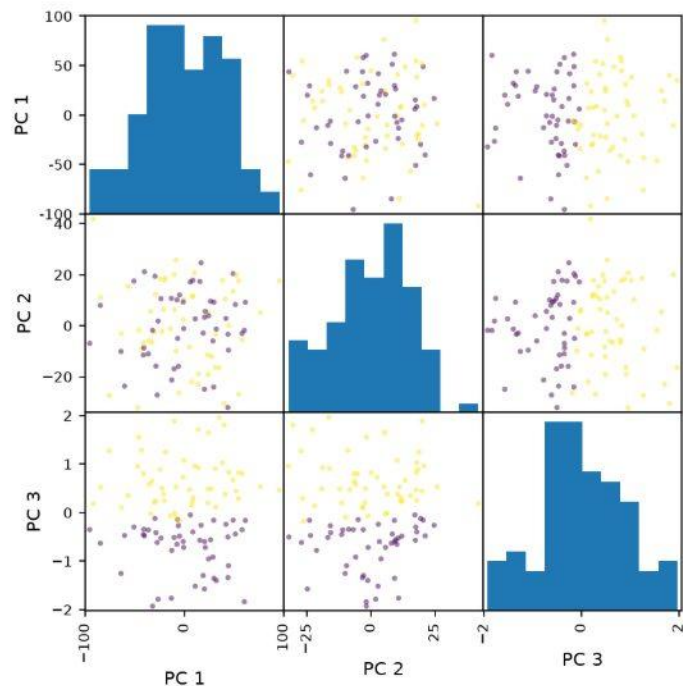
`PCA(n_components=0.95)` # keep as many components as needed for 95% of the variance (here: 10)

PCA is Unsupervised!

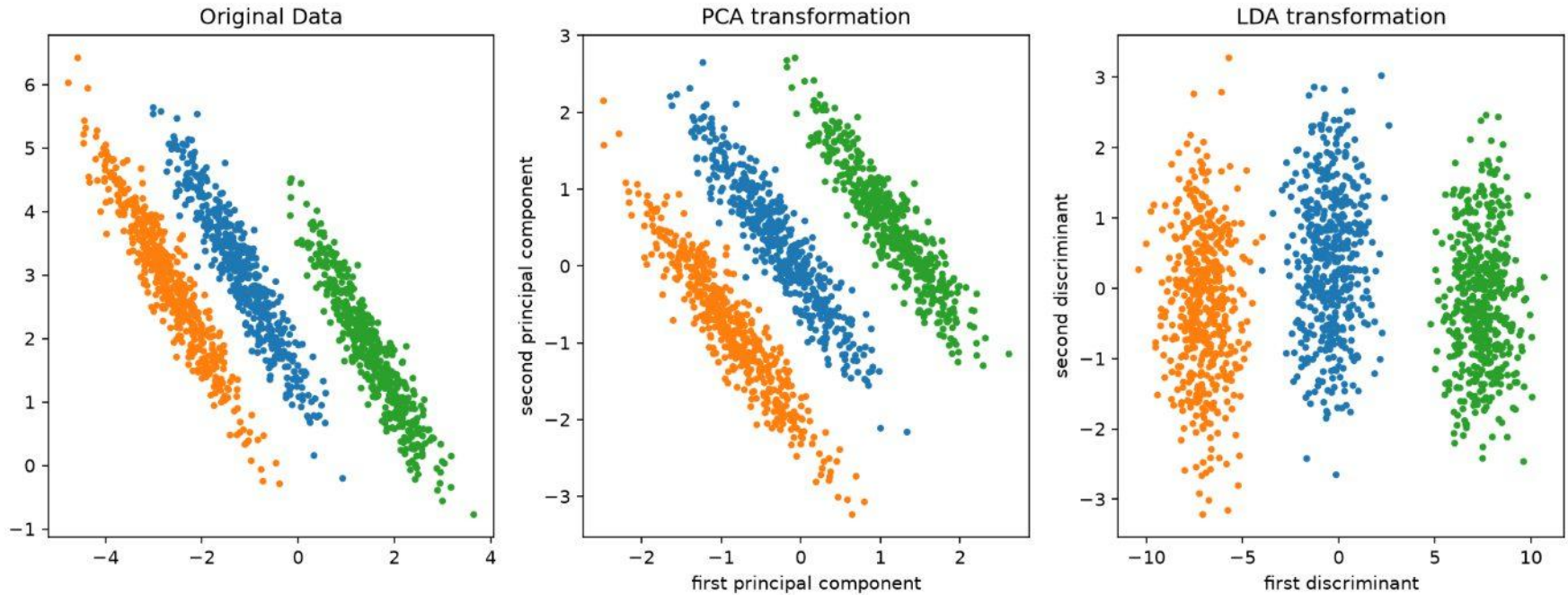
Data



Principal Components



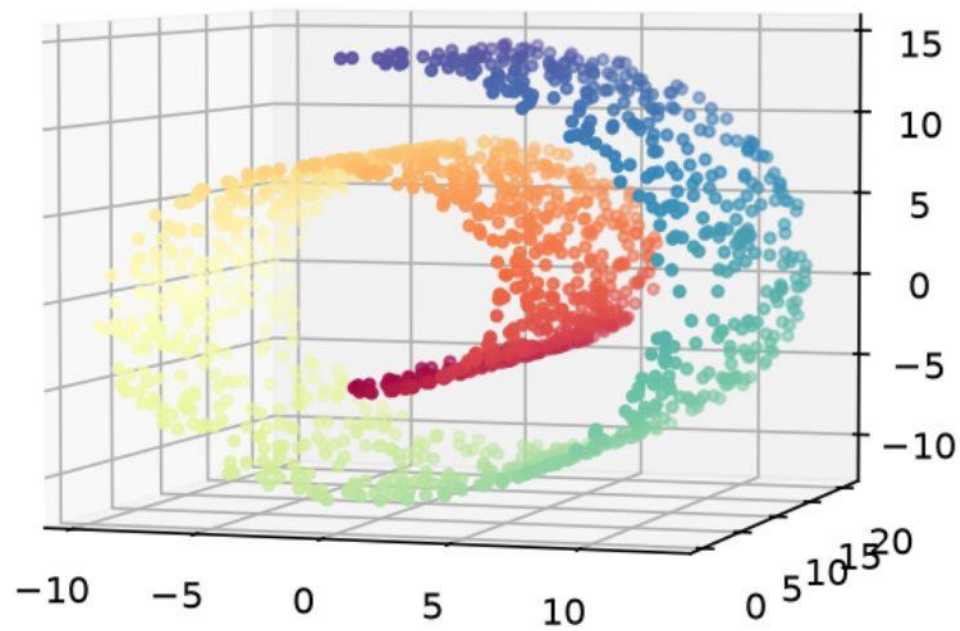
PCA vs Linear Discriminants



Summary: linear methods

- PCA: visualization, exploring correlations, compression, denoising
- PCA can help as regularization or feature extraction, but it ignores the target
- Scale features first unless they share a meaningful unit
- LDA: supervised linear projection, at most `n_classes - 1` dimensions

Manifold Learning



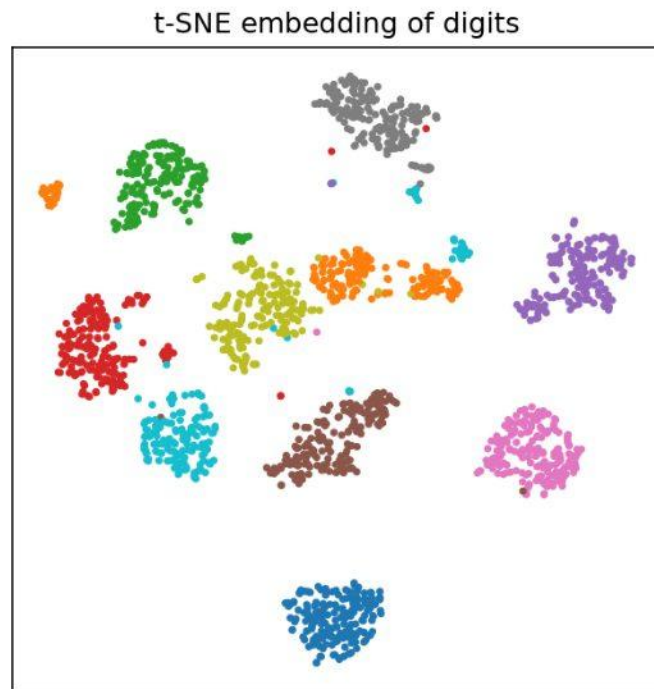
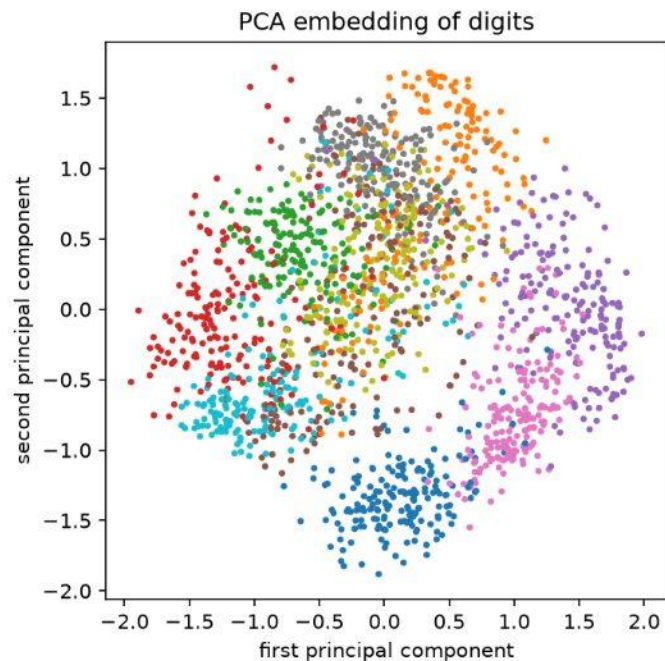
Pros and Cons

- Mostly for visualization and exploration
- Axes do not correspond to anything in the input space
- Many methods cannot embed new points (scikit-learn's TSNE has no transform)
- UMAP can transform new points, but is still best treated as exploratory
- Results depend on hyperparameters and random seeds

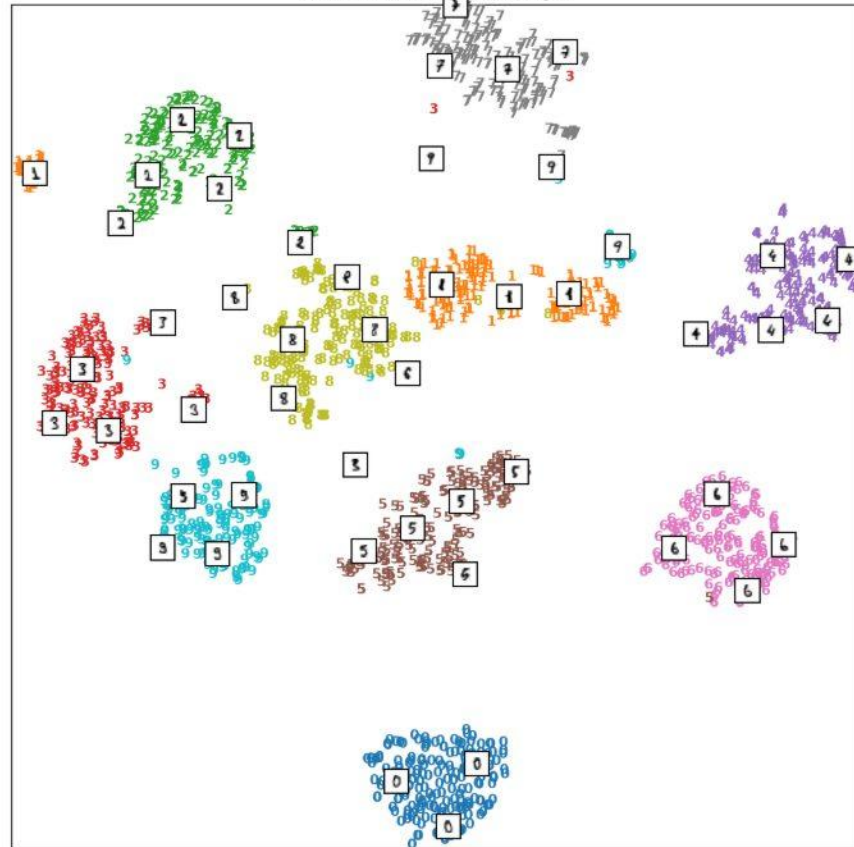
t-SNE in practice

- Directly optimizes 2D positions y_i by gradient descent (no mapping is learned)
- Local neighborhoods matter; global distances much less
- Heavy-tailed Student-t in 2D avoids crowding
- Perplexity sets σ_i : the effective number of neighbors

```
from sklearn.manifold import TSNE
from sklearn.datasets import load_digits
digits = load_digits()
X = digits.data / 16.
X_tsne = TSNE(random_state=0).fit_transform(X) # defaults: init="pca", learning_r
X_pca = PCA(n_components=2).fit_transform(X)
```

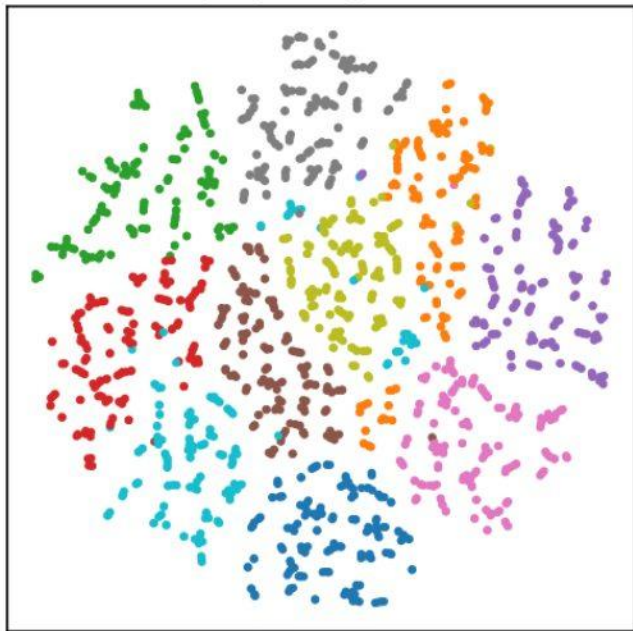


t-SNE embedding of the digits

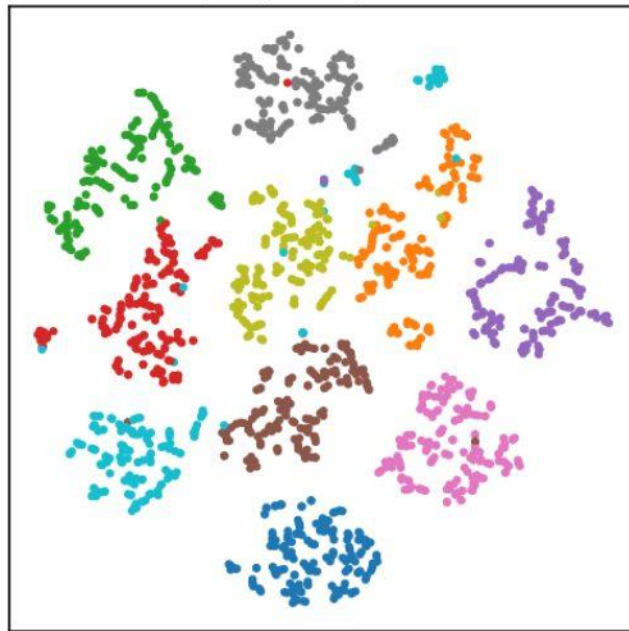


Tuning t-SNE perplexity: 2 and 5

perplexity = 2

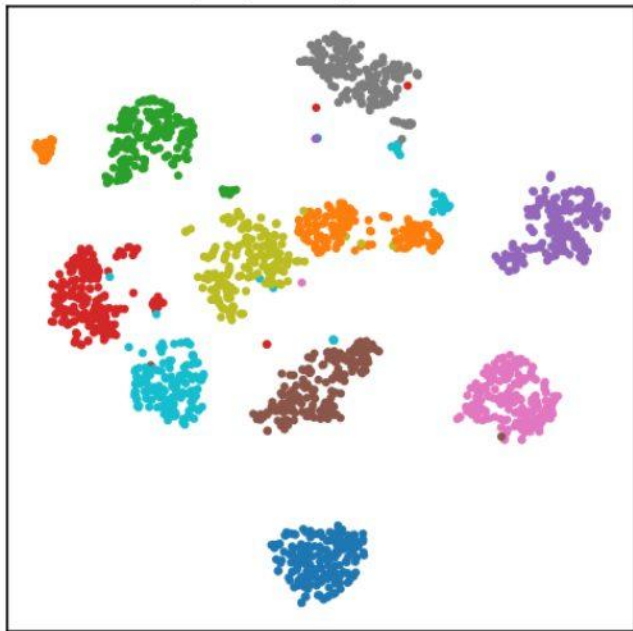


perplexity = 5

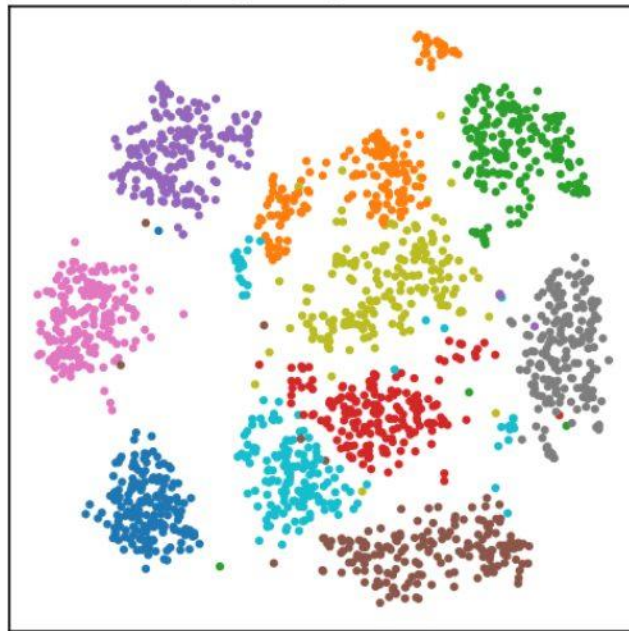


Tuning t-SNE perplexity: 30 and 300

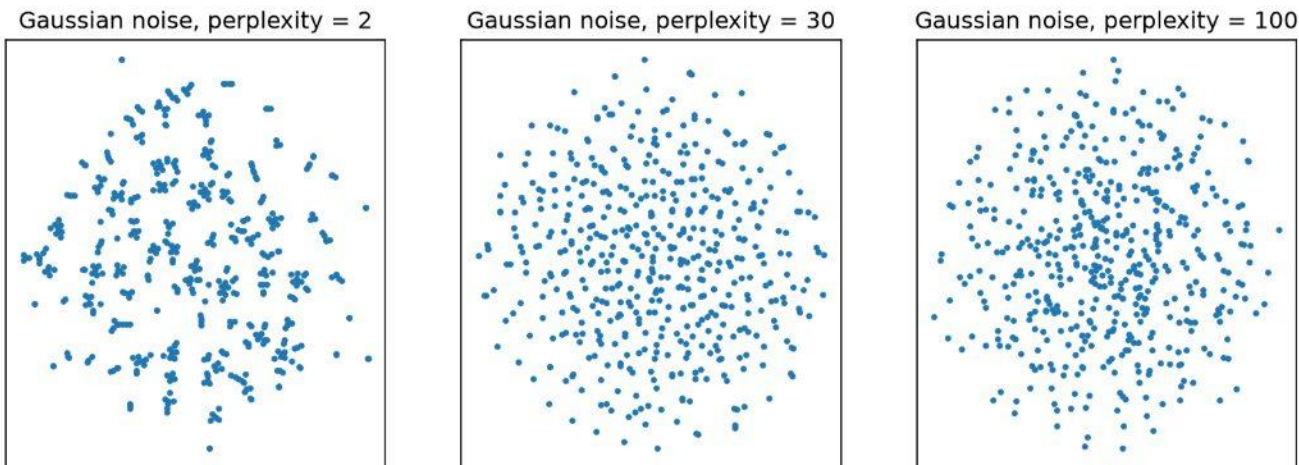
perplexity = 30



perplexity = 300



What goes wrong in practice: reading t-SNE plots



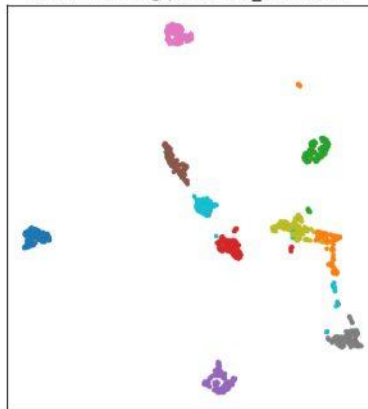
- Low perplexity creates "clusters" in pure Gaussian noise (left).
- Cluster **sizes** and **distances between clusters** in the plot are not meaningful.
- One seed, one perplexity is not an analysis. Compare several runs.
- Read more: <https://distill.pub/2016/misread-tsne/>

UMAP

- Build a weighted k-nearest-neighbor graph in input space
- Find a low-dimensional layout whose graph matches (cross-entropy)
- Usually faster than t-SNE on large data; often keeps more global structure
- Has `transform`: embeds new points

```
import umap # pip/uv package: umap-learn  
reducer = umap.UMAP(n_neighbors=15, min_dist=0.1,  
                    random_state=0)  
Z_train = reducer.fit_transform(X_train)  
Z_test = reducer.transform(X_test)
```

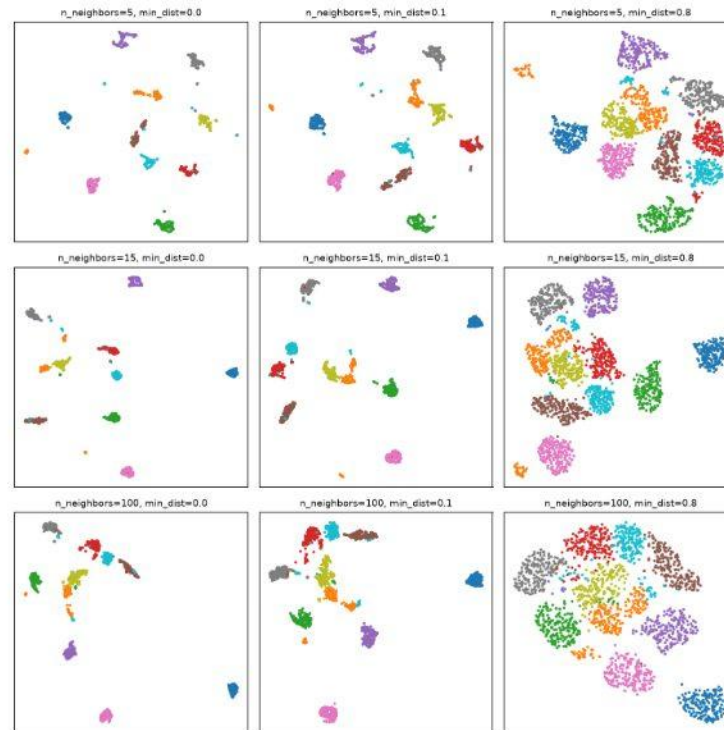
UMAP: training points (fit_transform)



UMAP: test points (transform)



UMAP hyperparameters



`n_neighbors`: local vs global structure (like perplexity) · `min_dist`: how tightly points may be packed

PCA, t-SNE or UMAP?

	PCA	t-SNE	UMAP
linear	yes	no	no
deterministic	yes	no (seed)	no (seed)
embeds new points	yes	no (scikit-learn)	yes
preserves	global variance	local neighborhoods	local, some global
speed, 100k points	seconds	minutes	about a minute
axes interpretable	yes (loadings)	no	no
use as model features	yes	no	with care

Try it 1: Which map keeps the neighbors?

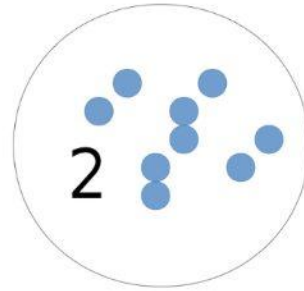
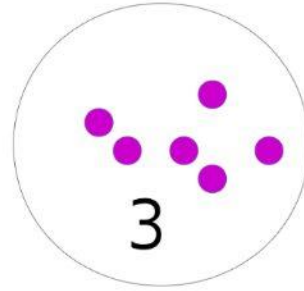
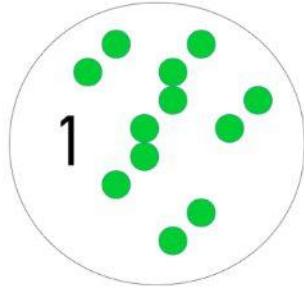
(15 min)

lab.ipynb, section **Try it 1** (digits data)

1. **Predict first:** rank PCA, t-SNE and UMAP by how well a 2D map preserves each point's 10 nearest neighbors. Write your ranking down.
2. Add t-SNE and UMAP to `embeddings` and run the cell. It reports
 - `trustworthiness(X, Z, n_neighbors=10)` (1.0 = neighbors in 2D were neighbors in 64D)
 - 5-fold CV accuracy of 1-NN on the 2D coordinates
3. Change `n_neighbors` to 50 in `trustworthiness`. Does the ranking change?
4. Discuss: why is 1-NN accuracy on a t-SNE map an optimistic estimate of how good a classifier would be?

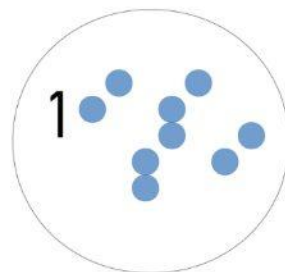
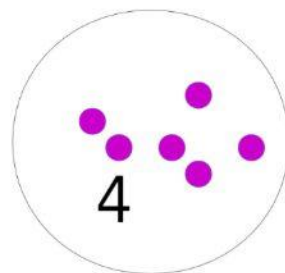
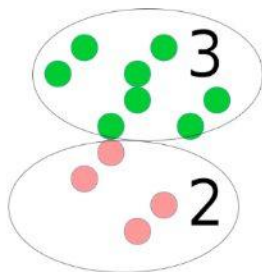
Break (10 min)

Part 2: Clustering and Mixture Models



Clustering

- Partition data into groups (clusters)
- Points within a cluster should be "similar"
- Points in different clusters should be "different"
- Number of clusters often has to be specified
- "Similar" is defined by the algorithm and by your features and metric



Goals of Clustering

- Data exploration
 - Are there coherent groups? How many?
- Data partitioning
 - Divide data by group before further processing (e.g. one model per segment)
- Unsupervised feature extraction
 - Derive features from cluster membership or distances
- Evaluation and parameter tuning
 - Quantitative measures exist but are of limited use
 - Qualitative inspection is common
 - Best: a downstream task. What is your goal?

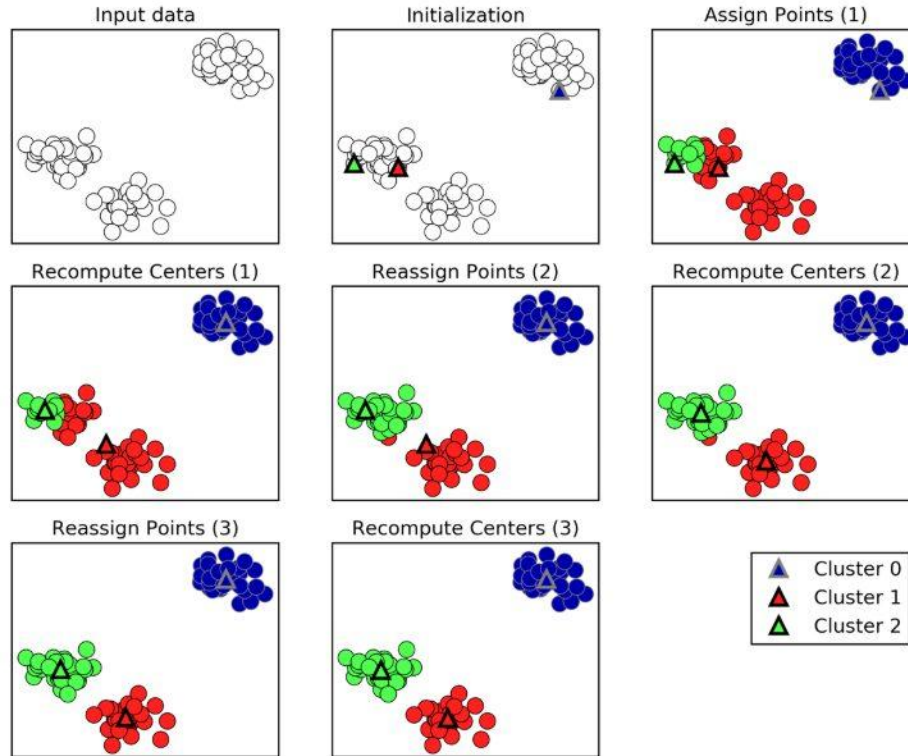
K-Means

Objective function for K-Means

$$\min_{\mathbf{c}_j \in \mathbf{R}^p, j=1, \dots, k} \sum_i ||\mathbf{x}_i - \mathbf{c}_{x_i}||^2$$

$\mathbf{c}_{x_i}$ is the cluster center $\mathbf{c}_j$ closest to $\mathbf{x}_i$

K-Means algorithm (Lloyd's)



- Pick number of clusters k .
- Pick k initial centers.
- While cluster centers change:
 1. Assign each point to its closest center.
 2. Recompute each center as the mean of its points.

K-Means API

```
X, y = make_blobs(centers=4, random_state=1)
```

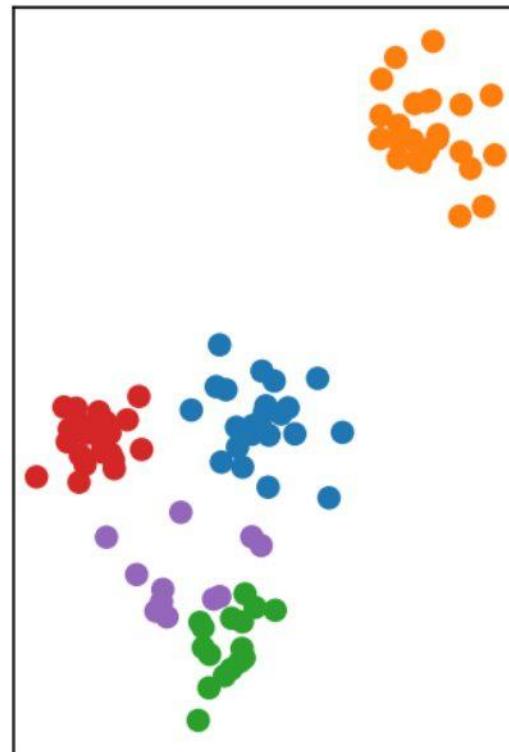
```
km = KMeans(n_clusters=5, random_state=0)  
km.fit(X)  
print(km.cluster_centers_.shape)  
print(km.labels_.shape)
```

```
(5, 2)  
(100,)
```

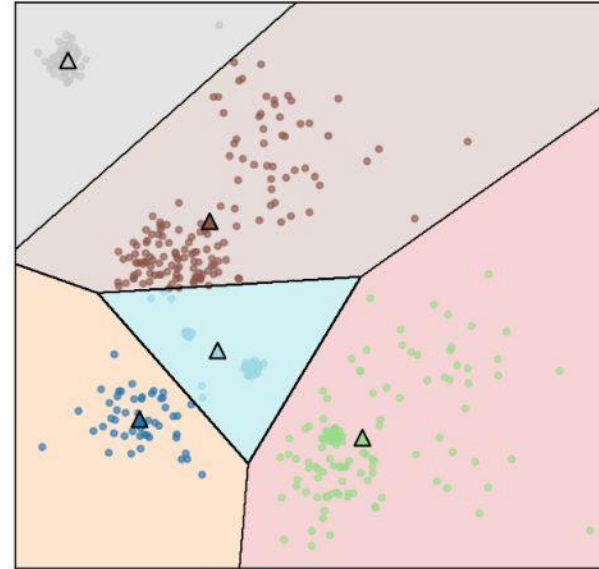
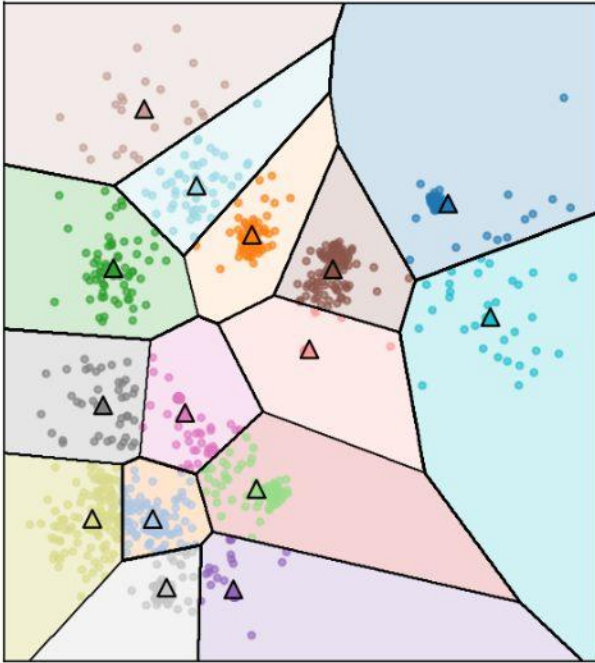
```
# predict is the same as labels_ on training data  
# but can be applied to new data  
print(km.predict(X).shape)
```

```
(100,)
```

```
plt.scatter(X[:, 0], X[:, 1], c=km.labels_)
```

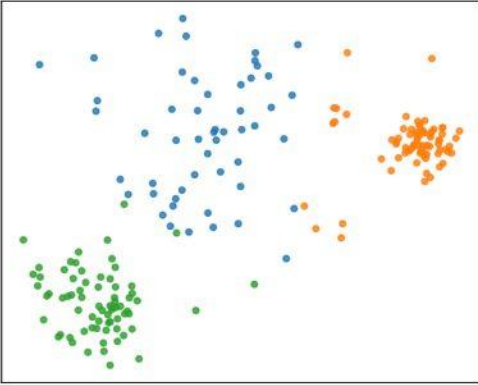


Restriction of Cluster Shapes



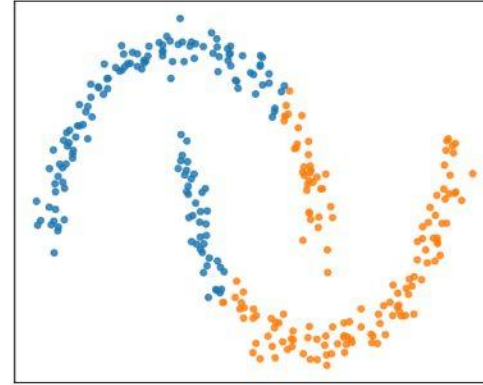
Clusters are Voronoi cells of the centers: always convex

Limitations of K-Means



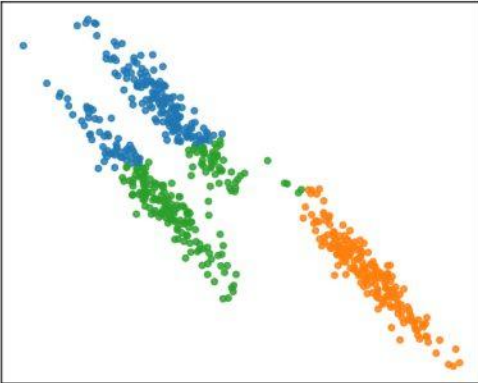
Boundaries

equidistant to centers: clusters of different spread



shapes

Only simple (convex)



Cannot model

covariance

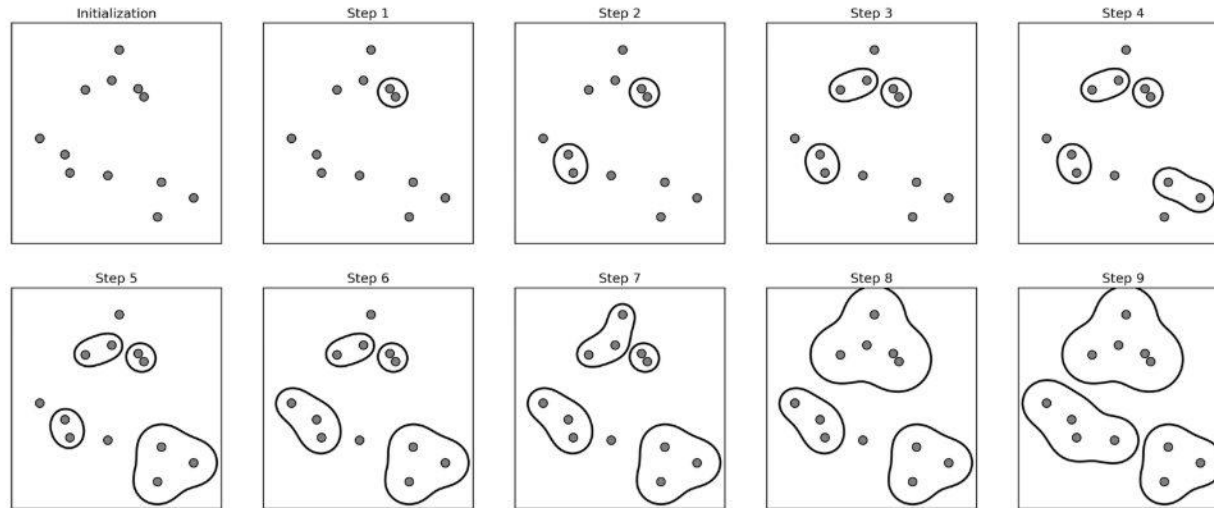
Initialization

- Random centers: fast but poor local optima
- k-means++ (default): sample new centers with probability $\propto$ squared distance to existing ones
- `n_init="auto"` (default): 1 run with k-means++, 10 runs with random init; best inertia is kept
- For large data, k-means++ itself can be expensive; set `n_init` explicitly

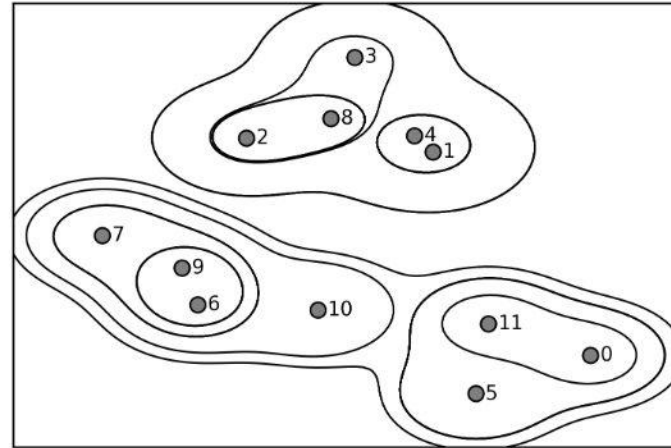
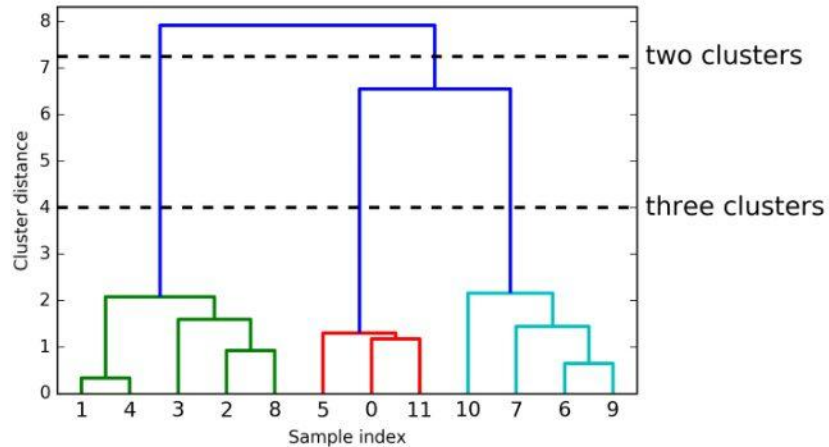
Agglomerative Clustering

Agglomerative Clustering

- Start with all points in their own cluster.
- Greedily merge the two most similar clusters.
- Result: a hierarchy of clusterings from n_{samples} clusters down to one.



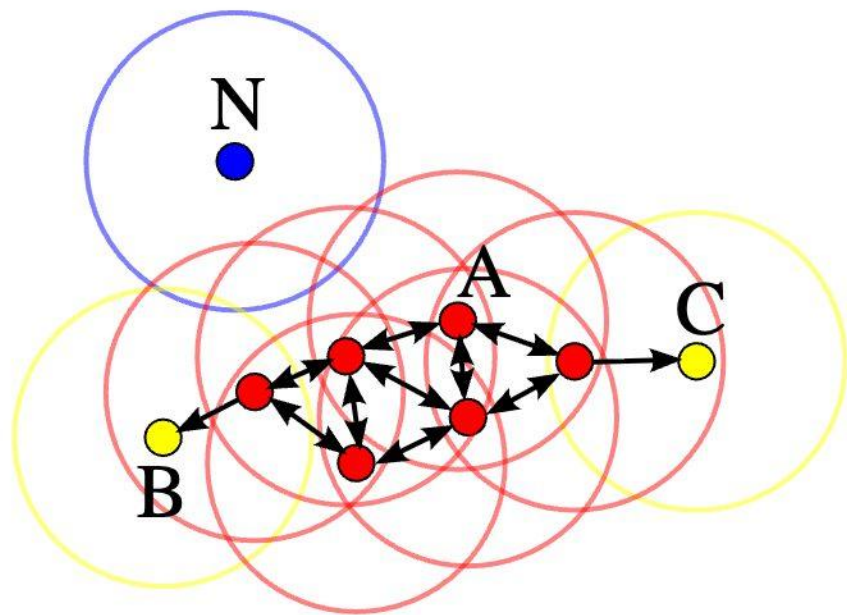
Dendrograms



```
from scipy.cluster.hierarchy import dendrogram, ward
dendrogram(ward(X))
```

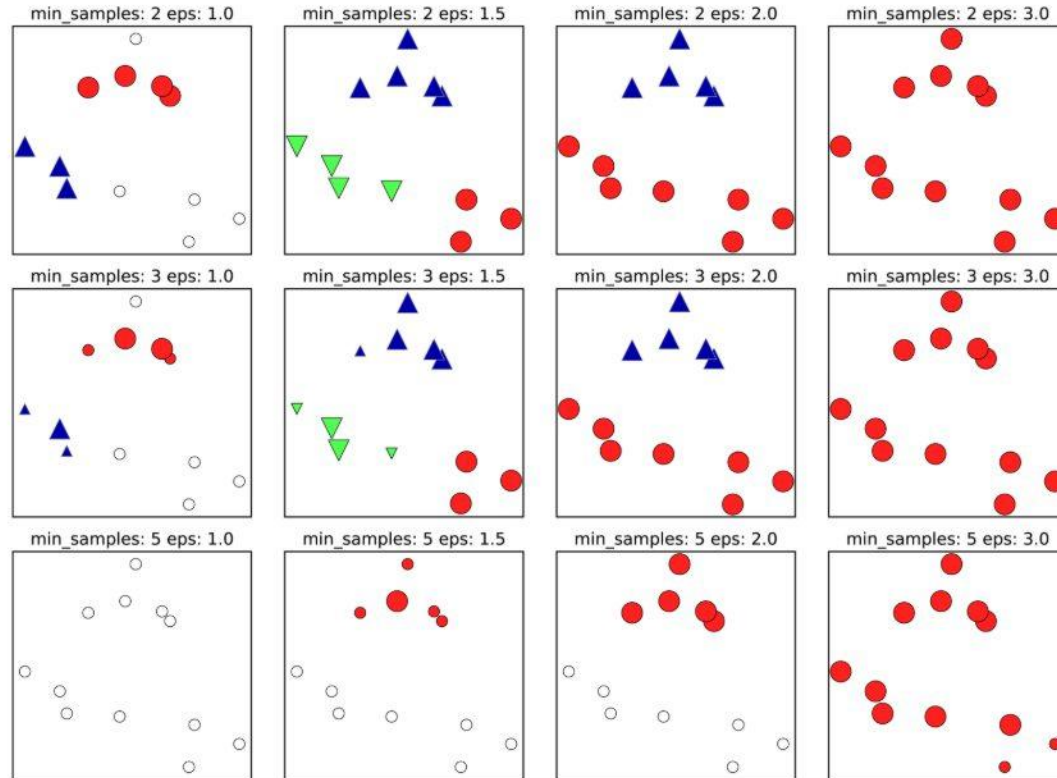
DBSCAN

Algorithm



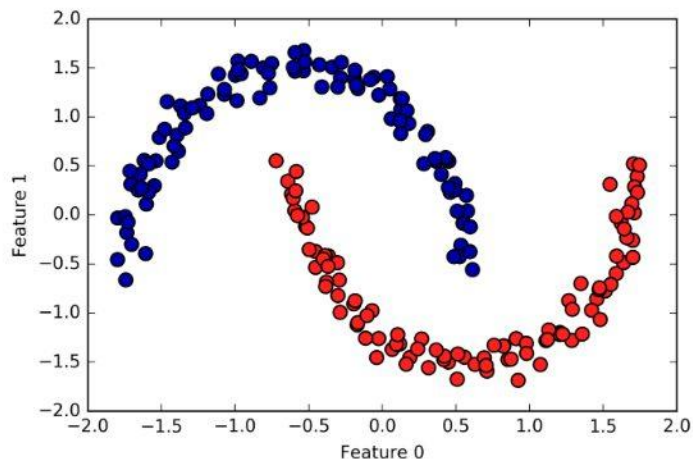
- eps: neighborhood radius
- min_samples: 4
- A: core point
- B, C: reachable, not core (border points)
- N: noise

Illustration of Parameters



Pros and Cons

- Pro: arbitrary cluster shapes
- Pro: detects noise points
- Pro: fast with a spatial index
- Con: two parameters, and ϵ is a single global density threshold
- Con: no `predict` for new points



HDBSCAN

- Runs DBSCAN for **all** eps at once (hierarchy of density levels)
- Keeps the most stable clusters across levels
- Main parameter: `min_cluster_size` (smallest group you care about)
- Handles clusters of different density; labels noise as -1
- `sklearn.cluster.HDBSCAN` (since 1.3)

Typical use:

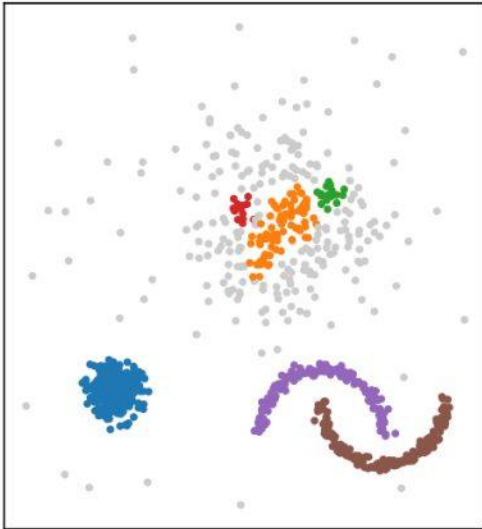
- exploratory clustering when k is unknown
- clustering UMAP embeddings (with small `min_dist`)
- when outliers must not be forced into a cluster

```
from sklearn.cluster import HDBSCAN

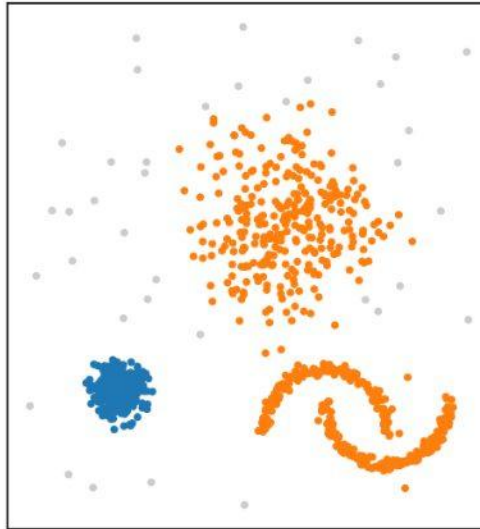
hdb = HDBSCAN(min_cluster_size=25, copy=True).fit(X)
hdb.labels_      # -1 = noise
hdb.probabilities_ # strength of membership
```

DBSCAN vs HDBSCAN: clusters of different density

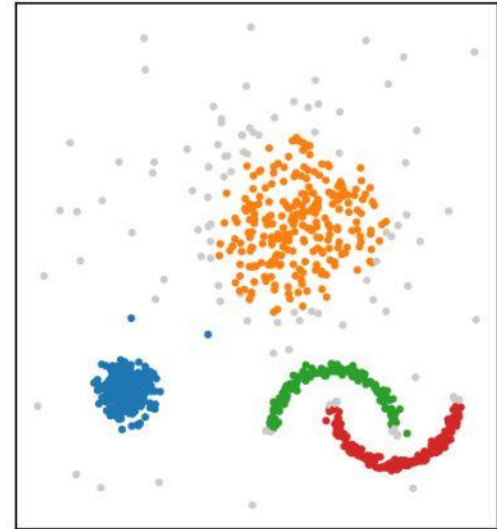
DBSCAN $\text{eps}=0.3$
6 clusters, 24% noise



DBSCAN $\text{eps}=0.8$
2 clusters, 4% noise



HDBSCAN $\text{min_cluster_size}=25$
4 clusters, 11% noise



(Gaussian) Mixture Models

Mixture Models

- Generative model: find $p(\mathbf{X})$.

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k p_k(\mathbf{x}|\theta_k)$$

- Data is a mixture of a small number of simple distributions
- Each point comes from one component
- Learn the component parameters θ_k and weights π_k

Gaussian Mixture Models

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \square(\mathbf{x}; \mu_k, \Sigma_k)$$

$$z \sim \text{Categorical}(\pi), \quad \mathbf{x} \mid z = k \sim \square(\mu_k, \Sigma_k)$$

- Non-convex likelihood: initialized with k-means, optional restarts (`n_init`)
- Optimization with the EM algorithm:
 - E-step: compute responsibilities $r_{ik} = P(z_i = k \mid \mathbf{x}_i)$ (soft assignments)
 - M-step: update π_k, μ_k, Σ_k as responsibility-weighted averages
 - Iterate until the log-likelihood stops improving

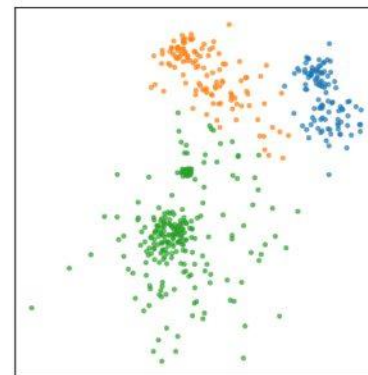
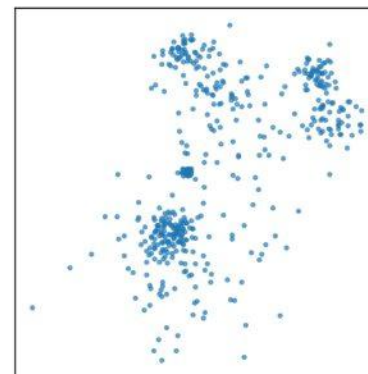
Why Mixture Models?

- Clustering (each component is a cluster), with soft assignments
- Parametric density model: how likely is a new point?
 - outlier / novelty scores (Week 12)
 - generative classifiers: one mixture per class
- Sampling new synthetic points

Examples

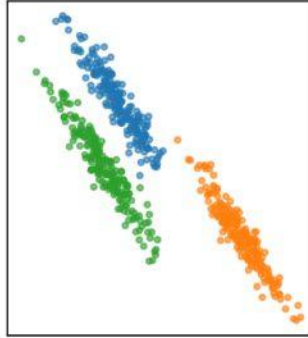
```
from sklearn.mixture import GaussianMixture
gmm = GaussianMixture(n_components=3, random_state=0)
gmm.fit(X)
print(gmm.means_)
print(gmm.covariances_)
```

```
[[ 8.701  5.21 ]
 [-0.282  6.829]
 [-2.331 -4.745]]
[[[ 1.44  -1.108]
  [-1.108  4.187]]
 [[ 5.836 -3.678]
  [-3.678  5.137]]
 [[ 6.441  1.789]
  [ 1.789 13.352]]]
```

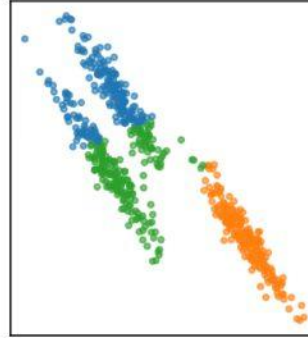


GMM vs KMeans

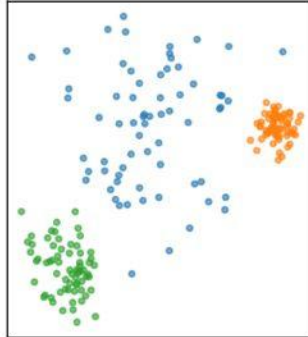
GaussianMixture



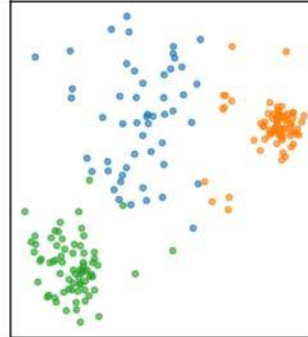
KMeans



GaussianMixture



KMeans

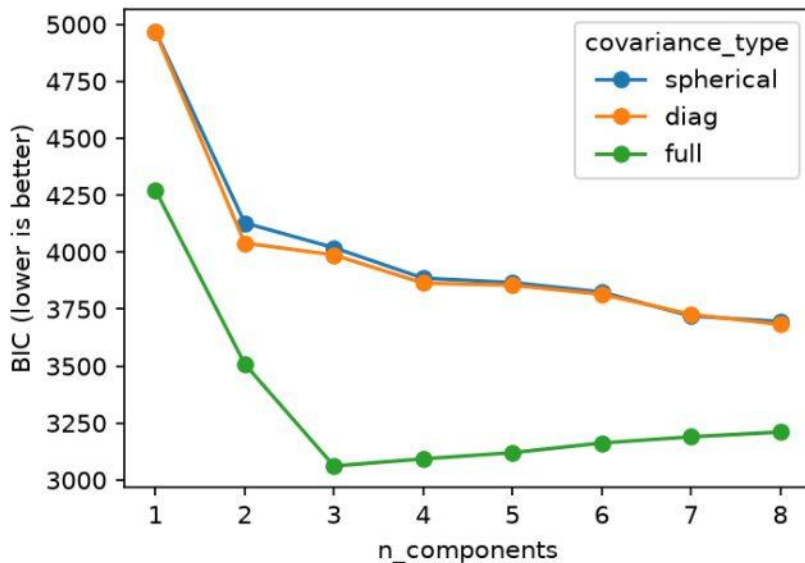


Choosing the number of components: BIC

$$\text{BIC} = -2 \log \hat{L} + d \log n$$

- d : number of free parameters, n : number of samples
- lower is better; penalizes extra components and covariance parameters

```
bics = [GaussianMixture(n, covariance_type="full", n_init=3,  
                        random_state=0).fit(X).bic(X)  
        for n in range(1, 9)]
```



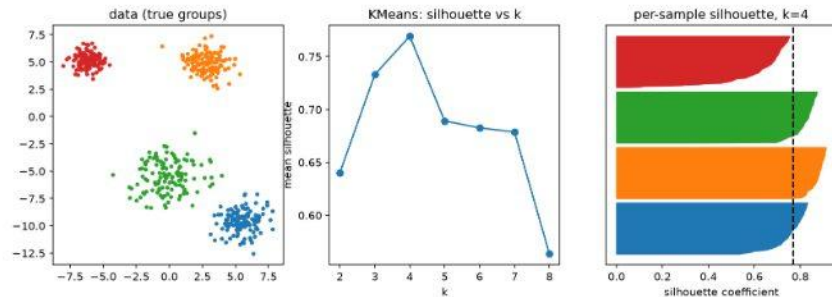
Evaluating Clusterings

Internal: silhouette coefficient

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1]$$

- $a(i)$: mean distance to points in its own cluster
- $b(i)$: mean distance to points in the nearest other cluster
- `silhouette_score` = mean of $s(i)$; also `davies_bouldin_score`, `calinski_harabasz_score`

```
from sklearn.metrics import silhouette_score
for k in range(2, 9):
    labels = KMeans(n_clusters=k, random_state=0).fit_predict(X)
    print(k, silhouette_score(X, labels))
```



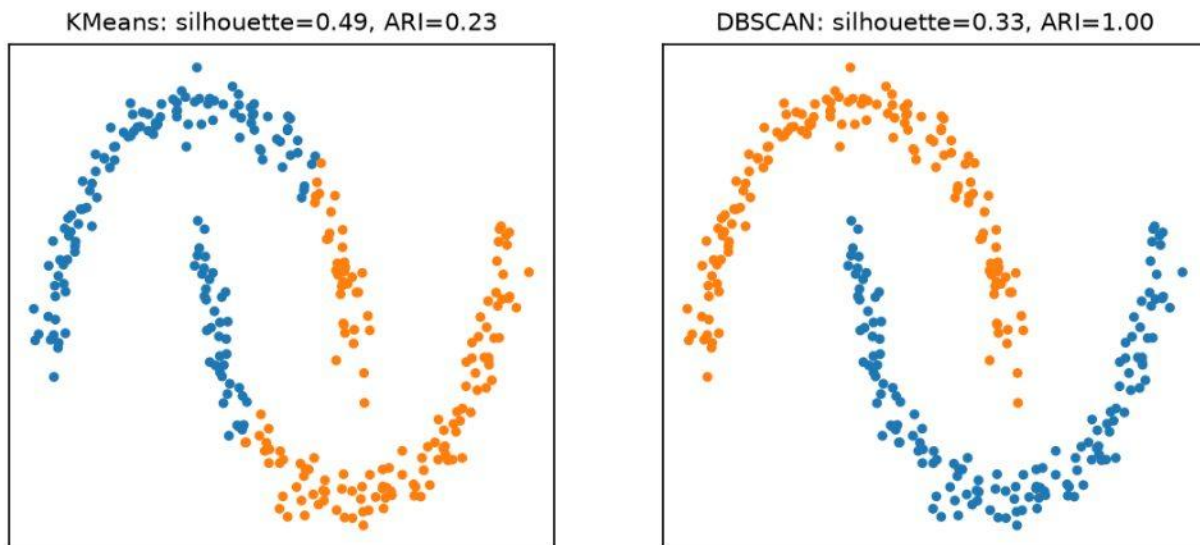
External: ARI and AMI (when labels exist)

- Compare a clustering with a reference partition (true labels, or another clustering)
- Invariant to permuting cluster ids, adjusted so that random labelings score about 0
- **ARI** (adjusted Rand index): agreement over all pairs of points; 1 = identical partitions
- **AMI** (adjusted mutual information): shared information, adjusted for chance; less biased with many small clusters

```
from sklearn.metrics import adjusted_rand_score, adjusted_mutual_info_score
adjusted_rand_score(y_true, labels), adjusted_mutual_info_score(y_true, labels)
adjusted_rand_score([0, 0, 1, 1], [1, 1, 0, 0]) # 1.0: same partition, different ids
```

Uses: benchmarking algorithms on labeled data, checking stability across seeds or bootstrap samples, comparing to a known business category.

What goes wrong: internal scores have their own bias



- Silhouette rewards compact, convex, well-separated blobs: it prefers k-means here (0.49) over the correct DBSCAN solution (0.33, ARI = 1.00).
- Scores computed in different feature spaces (raw vs scaled vs log) are not comparable.
- A "best k" from silhouette, inertia elbow or BIC is a suggestion, not a fact.

Tracking a clustering sweep with MLflow

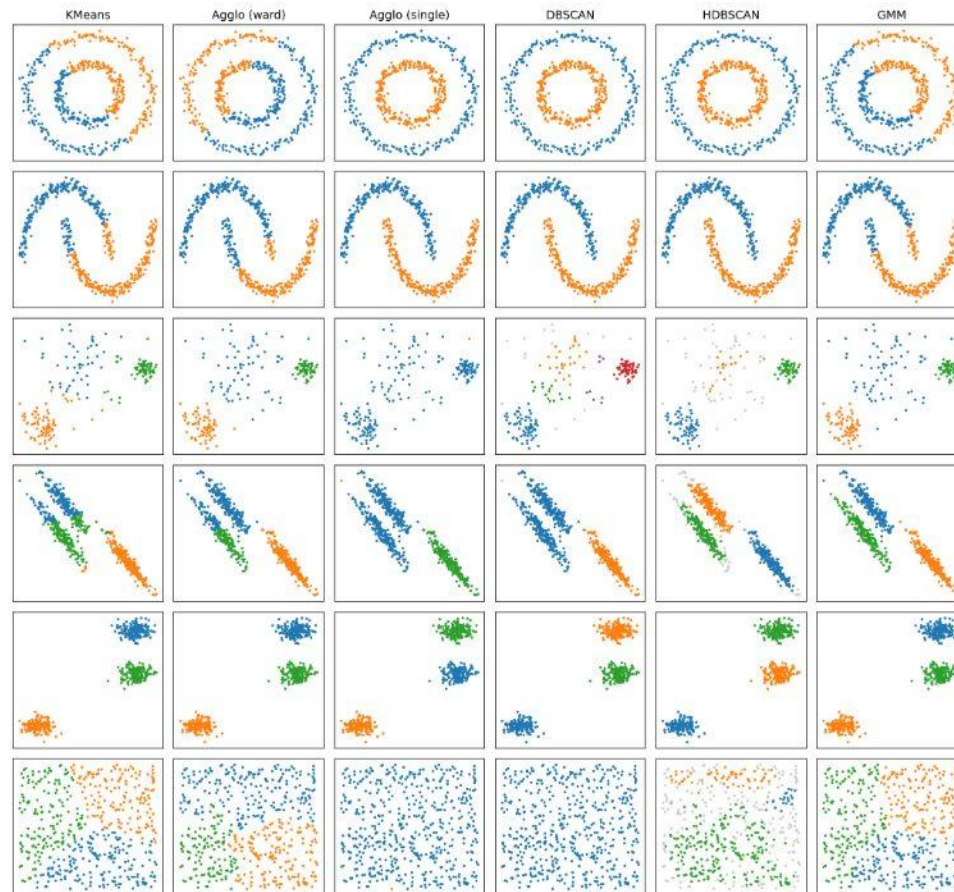
```
import mlflow
mlflow.set_experiment("week10-clustering")
for k in range(2, 9):
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    gmm = GaussianMixture(n_components=k, n_init=3, random_state=0).fit(X)
    with mlflow.start_run(run_name=f"k={k}"):
        mlflow.log_params({"k": k, "preprocessing": "log1p+standardize"})
        mlflow.log_metrics({"kmeans_silhouette": silhouette_score(X, km.labels_),
                           "kmeans_inertia": km.inertia_,
                           "gmm_bic": gmm.bic(X)})
runs = mlflow.search_runs(experiment_names=["week10-clustering"])
```

Wholesale customers, log + standardized: silhouette peaks at $k = 2$ (0.29), GMM BIC at $k = 3$. The criteria disagree, which is normal.

Try it 2: Segment the customers (15 min, teams of 2–3)

lab.ipynb, section **Try it 2** (wholesale customers)

1. Start from the baseline: raw spend, k-means, $k = 2$.
2. Choose preprocessing (raw / log / log + standardize), an algorithm (k-means, GMM, HDBSCAN, agglomerative) and its parameters.
3. Select using silhouette or BIC **and** whether you can describe each segment from the table of median spend per cluster.
4. Only at the end: compute ARI and AMI against the held-out `Channel` column; log your run to MLflow with your team name.
5. Be ready to describe your segments in one sentence each.



Summary

- **k-means:** classic, fast, convex clusters around centers; needs k
- **Agglomerative:** hierarchy and dendrogram, can use connectivity; Ward is a good default
- **DBSCAN / HDBSCAN:** arbitrary shapes, noise label; HDBSCAN handles varying density and needs no ϵ
- **Gaussian mixtures:** covariance, soft assignments, density model; choose K with BIC
- **Evaluation:** silhouette / BIC are biased guides; ARI / AMI need a reference; stability and usefulness decide

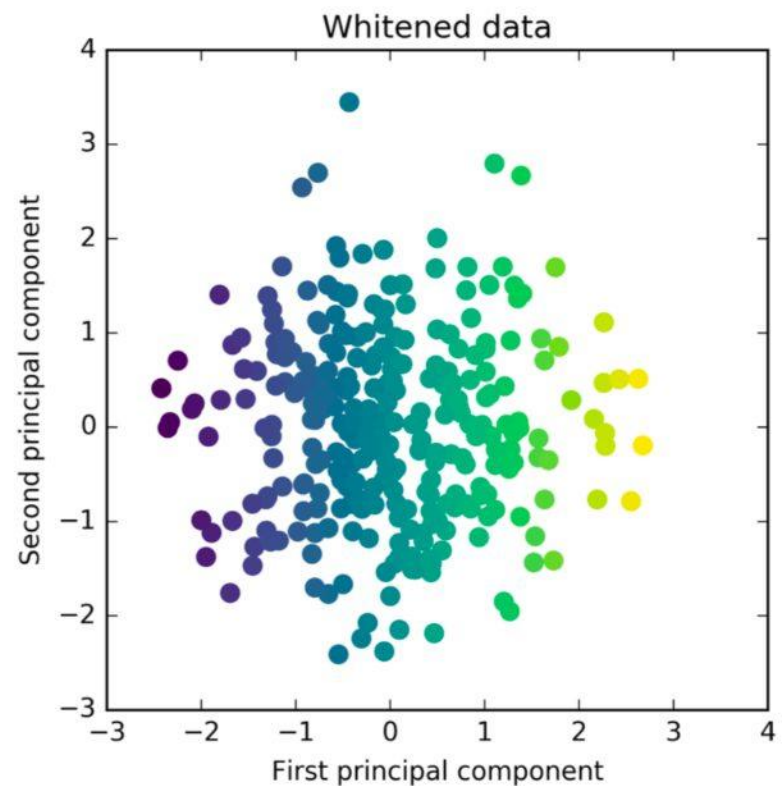
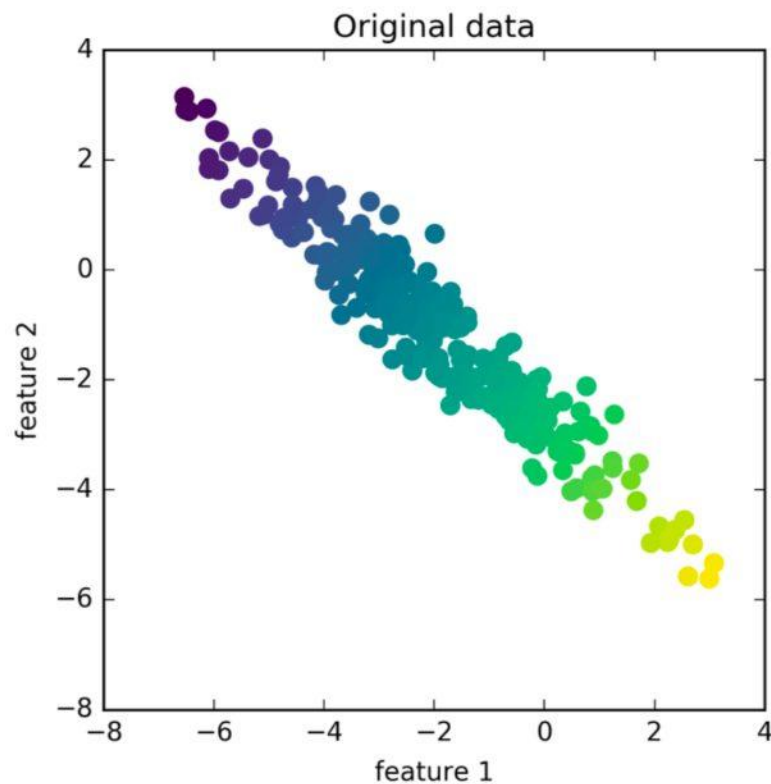
Next week and questions

- Lab: finish both "Try it" sections in `lab.ipynb`
- Week 11: time series forecasting
- Unsupervised ideas return in Week 12: anomaly detection and drift monitoring

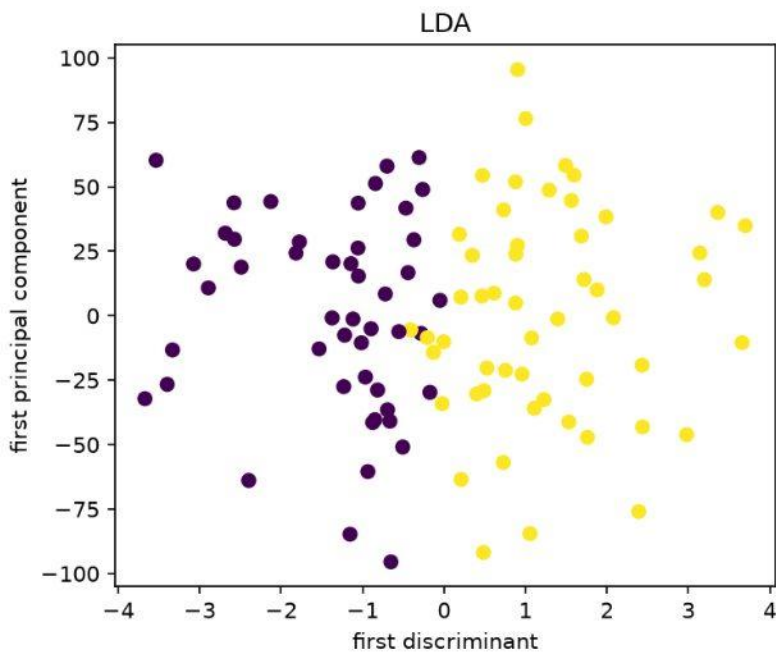
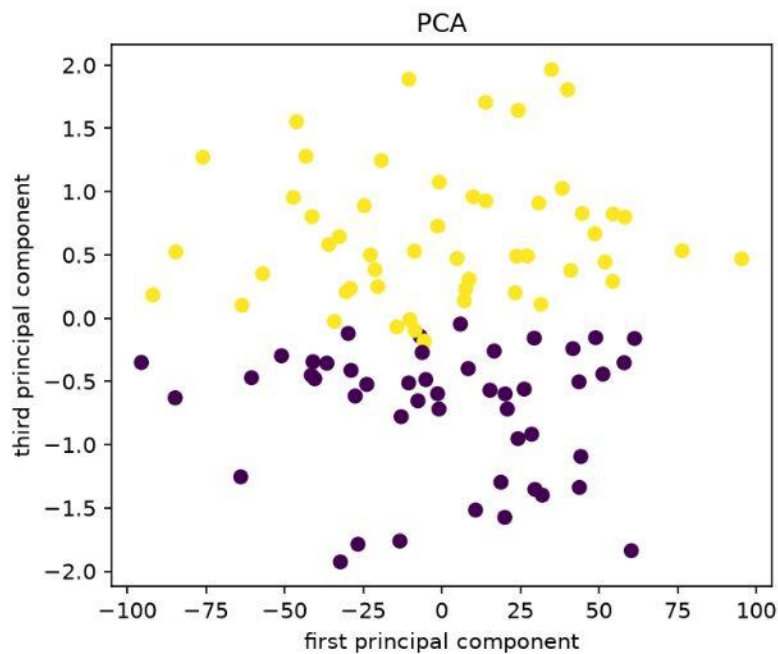
Questions?

Backup

Whitening



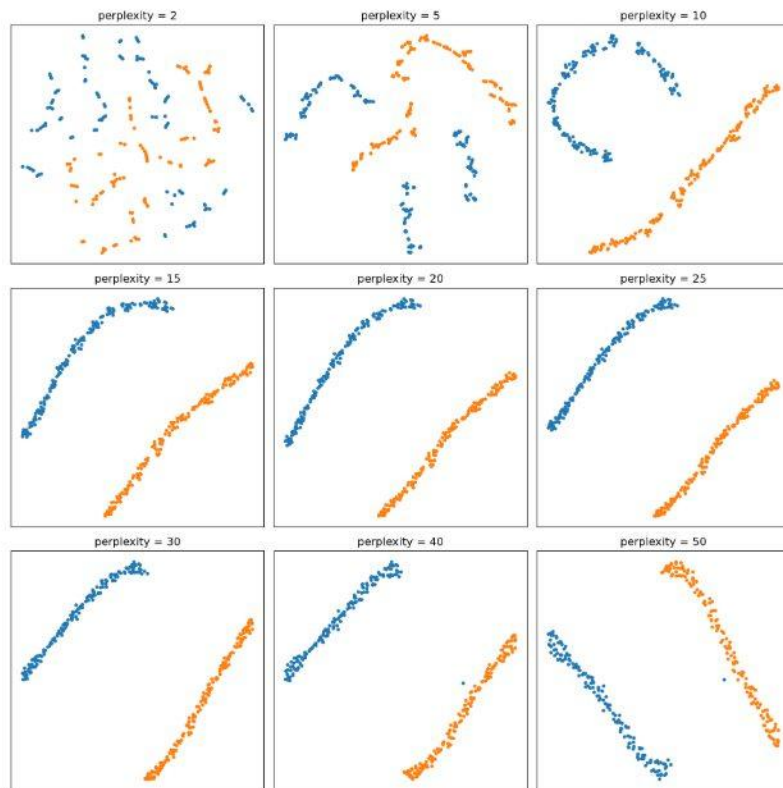
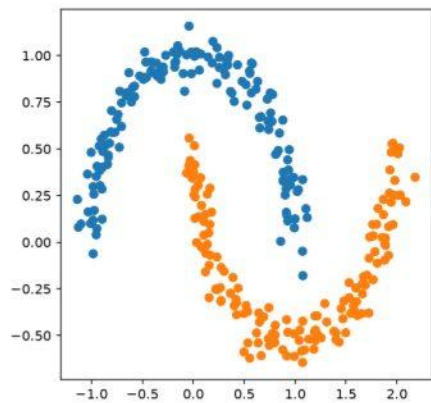
Data where PCA failed

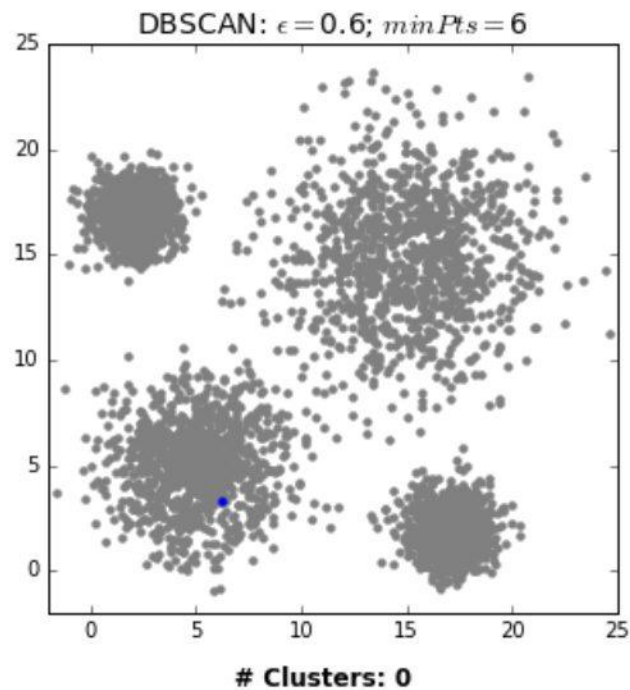
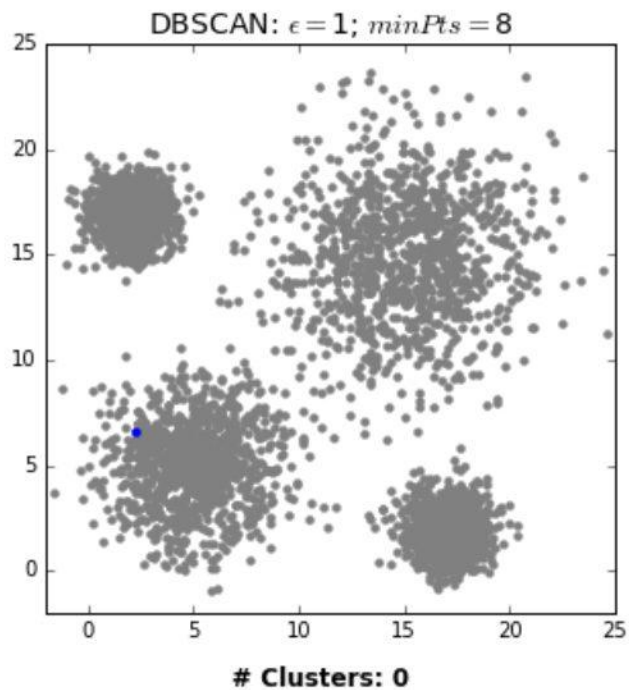


Showing algorithm progress

9

<https://github.com/oreillymedia/t-SNE-tutorial>





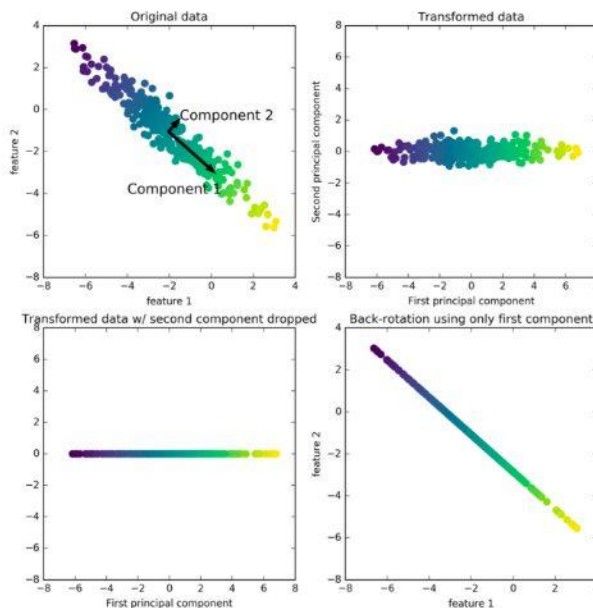
by David Sheehan [dashee87.github.io](https://github.com/dashee87)

PCA objective(s)

Maximum variance:

$$\max_{u_1 \in \mathbb{R}^p, \|u_1\|=1} \text{var}(Xu_1)$$

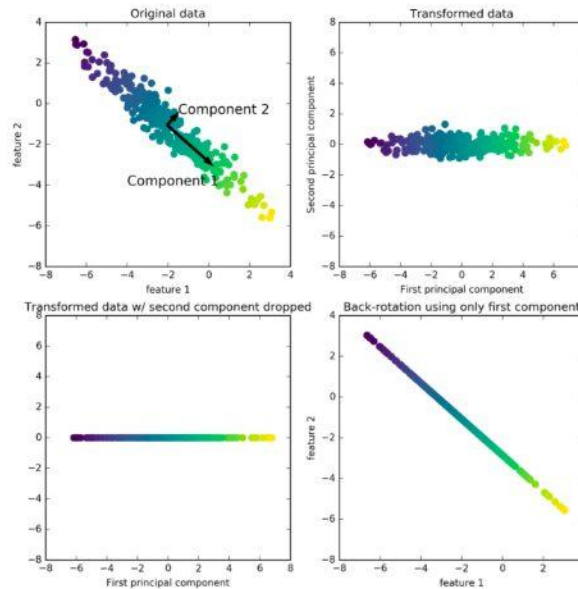
$$\max_{u_1 \in \mathbb{R}^p, \|u_1\|=1} u_1^T \text{cov}(X) u_1$$



PCA objective(s)

Minimum reconstruction error with a rank constraint:

$$\min_{X', \text{rank}(X')=r} \|X - X'\|_F^2$$



PCA Computation

- Center X (subtract mean).
- In practice: often also scale to unit variance.
- Compute singular value decomposition:

$$X = UDV^T$$

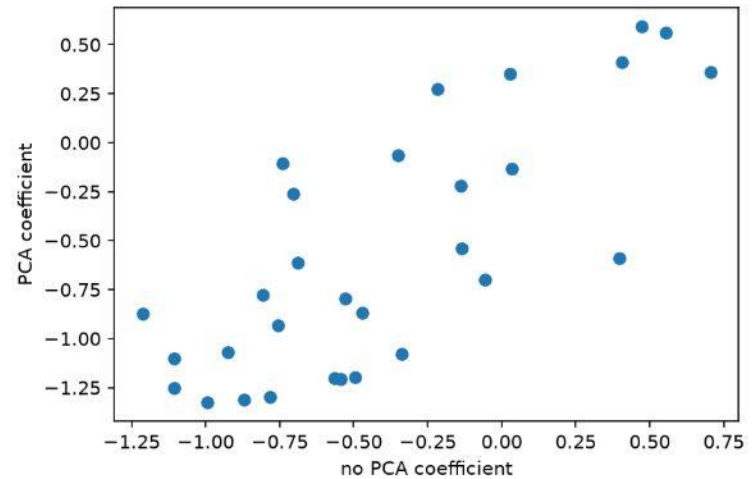
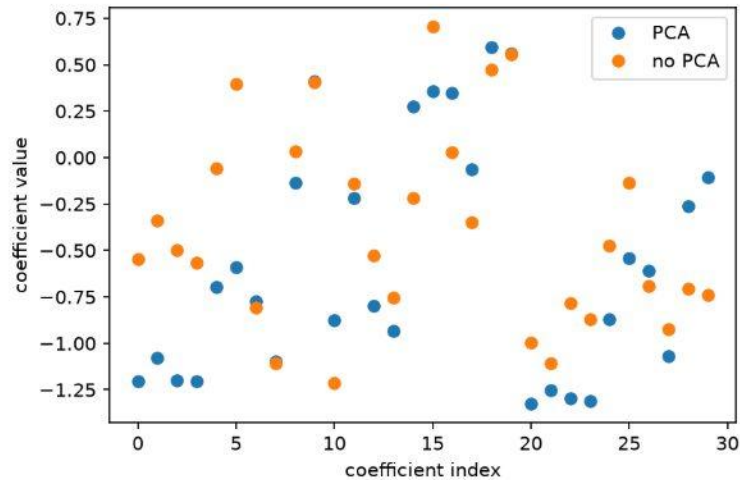
Diagram illustrating the Singular Value Decomposition (SVD) equation $X = UDV^T$:

- U : $n_{\text{samples}} \times n_{\text{samples}}$ orthogonal matrix (not necessarily computed in practice). An arrow points from this text to the U matrix in the equation.
- D : Diagonal (containing singular values). An arrow points from this text to the D matrix in the equation.
- V^T : $n_{\text{features}} \times n_{\text{features}}$ orthogonal matrix. Contains component vectors. Drop rows (of V^T) for dimensionality reduction. An arrow points from this text to the V^T matrix in the equation.

Interpreting coefficients

```
pca = pca_lr.named_steps['pca']  
lr = pca_lr.named_steps['logisticregression']  
coef_pca = lr.coef_ @ pca.components_ # rotate back into the (scaled) input space
```

Comparing PCA + logreg vs plain logreg:



Discriminant Analysis

Linear Discriminant Analysis aka Fisher Discriminant

$$P(y = k|X) = \frac{P(X|y = k)P(y = k)}{P(X)} = \frac{P(X|y = k)P(y = k)}{\sum_l P(X|y = l) \cdot P(y = l)}$$

$$p(X|y = k) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(X - \mu_k)^T \Sigma^{-1} (X - \mu_k)\right)$$

$$\log\left(\frac{P(y = k|X)}{P(y = l|X)}\right) = 0 \Leftrightarrow (\mu_k - \mu_l)^T \Sigma^{-1} X = \frac{1}{2}(\mu_k^T \Sigma^{-1} \mu_k - \mu_l^T \Sigma^{-1} \mu_l) + \log \frac{P(y = l)}{P(y = k)}$$

LDA in practice

- Generative model: each class is Gaussian, all classes share one covariance matrix.
- Fit = class means + pooled covariance; no hyperparameters to tune (except optional shrinkage).
- Decision boundary is linear: nearest centroid after transforming the space by $\Sigma^{-1/2}$.

Quadratic Discriminant Analysis

LDA:

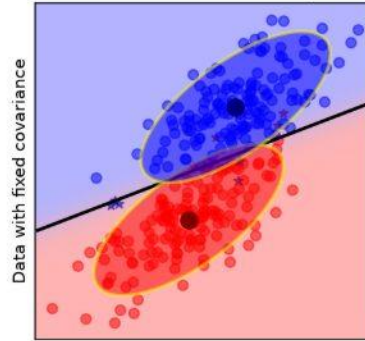
$$p(\mathbf{X} | y = k) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{X} - \mu_k)^T \Sigma^{-1} (\mathbf{X} - \mu_k)\right)$$

QDA:

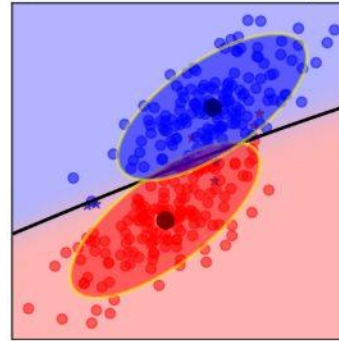
$$p(\mathbf{X} | y = k) = \frac{1}{(2\pi)^{p/2} |\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{X} - \mu_k)^T \Sigma_k^{-1} (\mathbf{X} - \mu_k)\right)$$

- Separate covariance per class: quadratic decision boundary.
- More flexible, but $\sim Kp^2/2$ covariance parameters: needs more data per class.
- Cannot be read as a single transformation of the space.

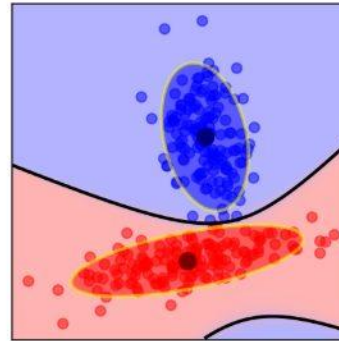
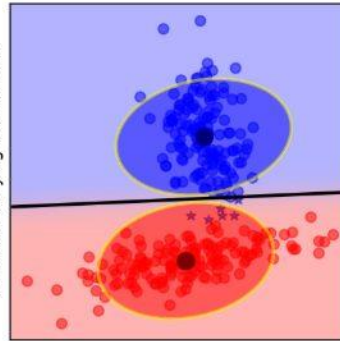
Linear Discriminant Analysis



Quadratic Discriminant Analysis



Data with varying covariances



Discriminants and PCA

- Both fit Gaussian models
- PCA: one Gaussian for the whole data
- LDA: one Gaussian per class, shared covariance
- LDA can transform the space:

```
LinearDiscriminantAnalysis(n_components=...).fit_transform(X, y)
```

- At most `n_classes - 1` components (the class means span that subspace)

Algorithms

- `KernelPCA`: PCA on a kernel matrix
- `SpectralEmbedding` (Laplacian eigenmaps): eigenvectors of the graph Laplacian
- `LocallyLinearEmbedding`, `Isomap` ("PCA on geodesic distances")
- `TSNE`: t-distributed stochastic neighbor embedding (scikit-learn; openTSNE for large data)
- `UMAP`: uniform manifold approximation and projection (`umap-learn`, scikit-learn API)

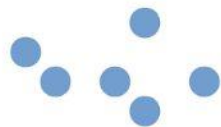
t-SNE

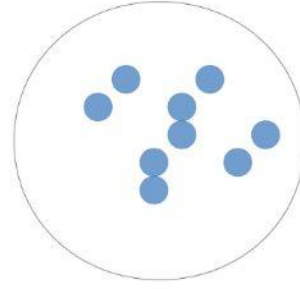
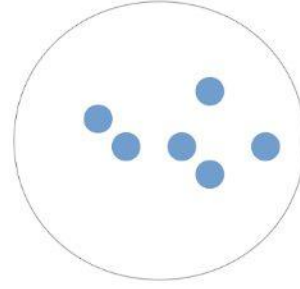
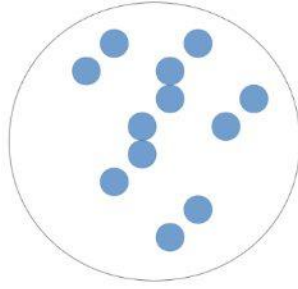
$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2 / 2\sigma_i^2)}$$

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$$

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|\mathbf{y}_i - \mathbf{y}_k\|^2)^{-1}}$$

$$\text{KL}(P||Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$





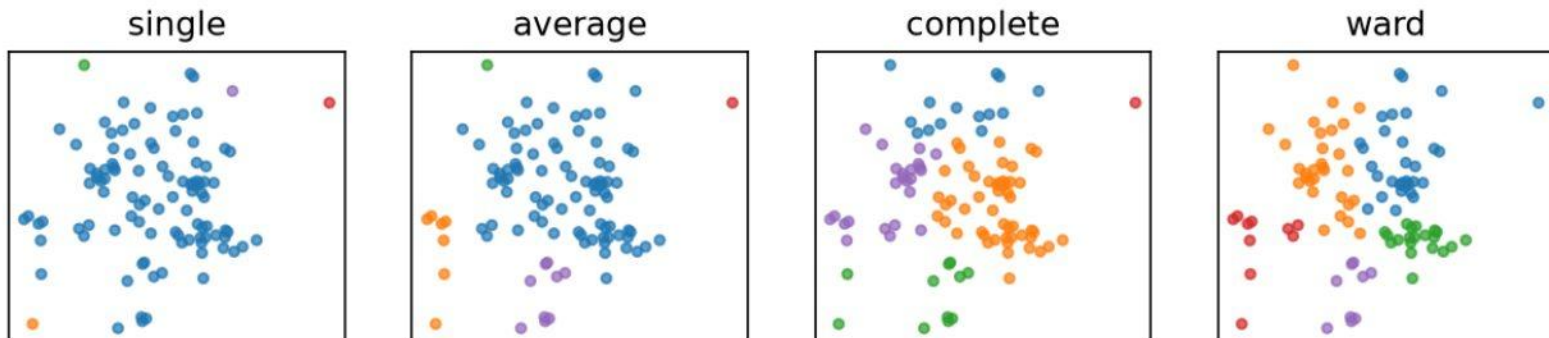
Computational Properties

- Naive implementation (Lloyd's):
 - $n_clusters * n_samples$ distance computations per iteration
- Fast "exact" algorithms:
 - Elkan's (triangle inequality), Yinyang, ...
- Approximate algorithms:
 - MiniBatchKMeans

Feature Extraction using K-Means

- Cluster membership → categorical feature
- Distances to centers → continuous features (`km.transform(X)`)
- Examples:
 - Partitioning a low-dimensional space (similar to basis functions)
 - Historically: image patch features ("bag of visual words", Coates & Ng 2011)
 - Vector quantization, and product quantization for fast approximate nearest-neighbor search

Linkage Criteria



Cluster sizes:

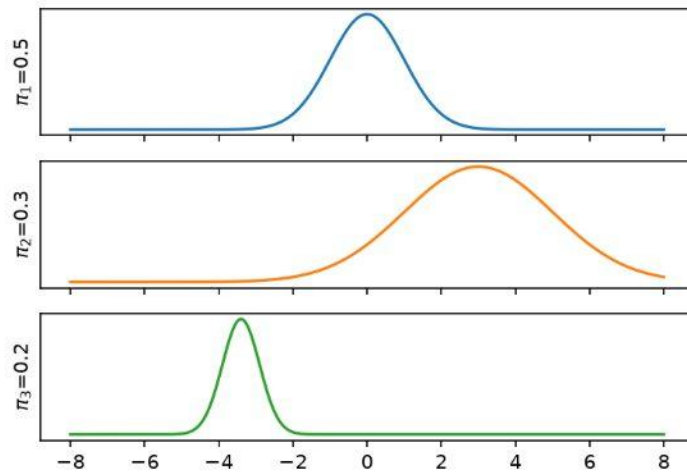
single	:	[96	1	1	1	1]
average	:	[82	9	7	1	1]
complete	:	[50	24	14	11	1]
ward	:	[31	30	20	10	9]

- Single linkage: smallest minimum distance
- Average linkage: smallest average distance between all pairs
- Complete linkage: smallest maximum distance
- Ward (default in scikit-learn): smallest increase in within-cluster variance, leads to more equally sized clusters

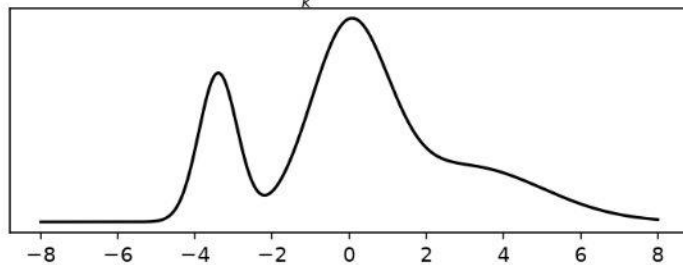
Pros and Cons

- Can restrict merges to a connectivity graph (e.g. k-NN graph, image grid)
- Fast with sparse connectivity, otherwise at least quadratic in n_{samples}
- Some linkage criteria lead to very imbalanced cluster sizes
- Hierarchy gives a more complete view than a single clustering

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \phi(\mathbf{x}; \mu_k, \Sigma_k)$$



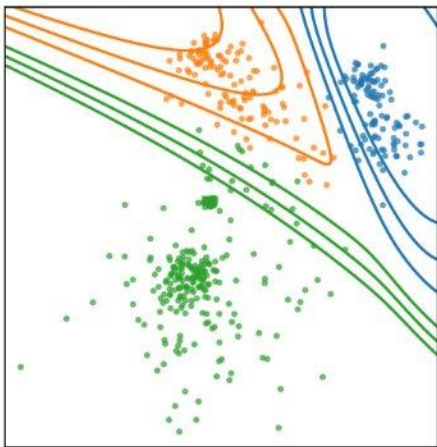
$$p(x) = \sum_k \pi_k \mathcal{N}(x; \mu_k, \sigma_k)$$



Probability Estimates

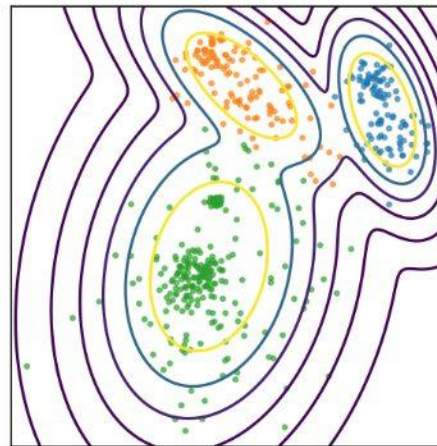
```
gmm.predict_proba(X)
```

```
array([[0., 0., 1.],  
       [1., 0., 0.],  
       ...])
```



```
# average log-likelihood per sample  
print(gmm.score(X))  
print(gmm.score_samples(X).shape)
```

```
-5.509  
(500,)
```



Covariance restrictions

